

EcomIQ Platform Manual

End-to-End System Architecture, Module Reference & Technology Rationale

Target Audience: Technical Developers, Architects, & Non-Technical Stakeholders

Author: EcomIQ Engineering Team | **Version:** 2.4.0 | **Date:** July 2026

Stack: Python · Apache Kafka · PySpark · PostgreSQL · FastAPI · FastMCP · Gemini AI · Docker

EXECUTIVE SUMMARY & SYSTEM PURPOSE

EcomIQ is a real-time, event-driven e-commerce analytics and operational intelligence platform. It continuously simulates, ingests, and processes streaming transactional events (orders, payments, shipments, returns, stock moves, user clicks) across a 14-table relational database schema. On top of this data pipeline, EcomIQ features an autonomous Multi-Agent AI System that converts plain English questions (e.g., 'Why are payments failing in West India?') into dynamic SQL filters, executes statistical anomaly detection across 17,200+ records, and synthesizes root-cause executive markdown reports in seconds.

1. High-Level Architecture & Block Diagram

The EcomIQ platform is built using a microservice container architecture hosted on an isolated Docker network named `ecom_iq_network`. Below is the full block diagram illustrating how synthetic data flows from generation to storage, and how user queries pass through the AI pipeline.

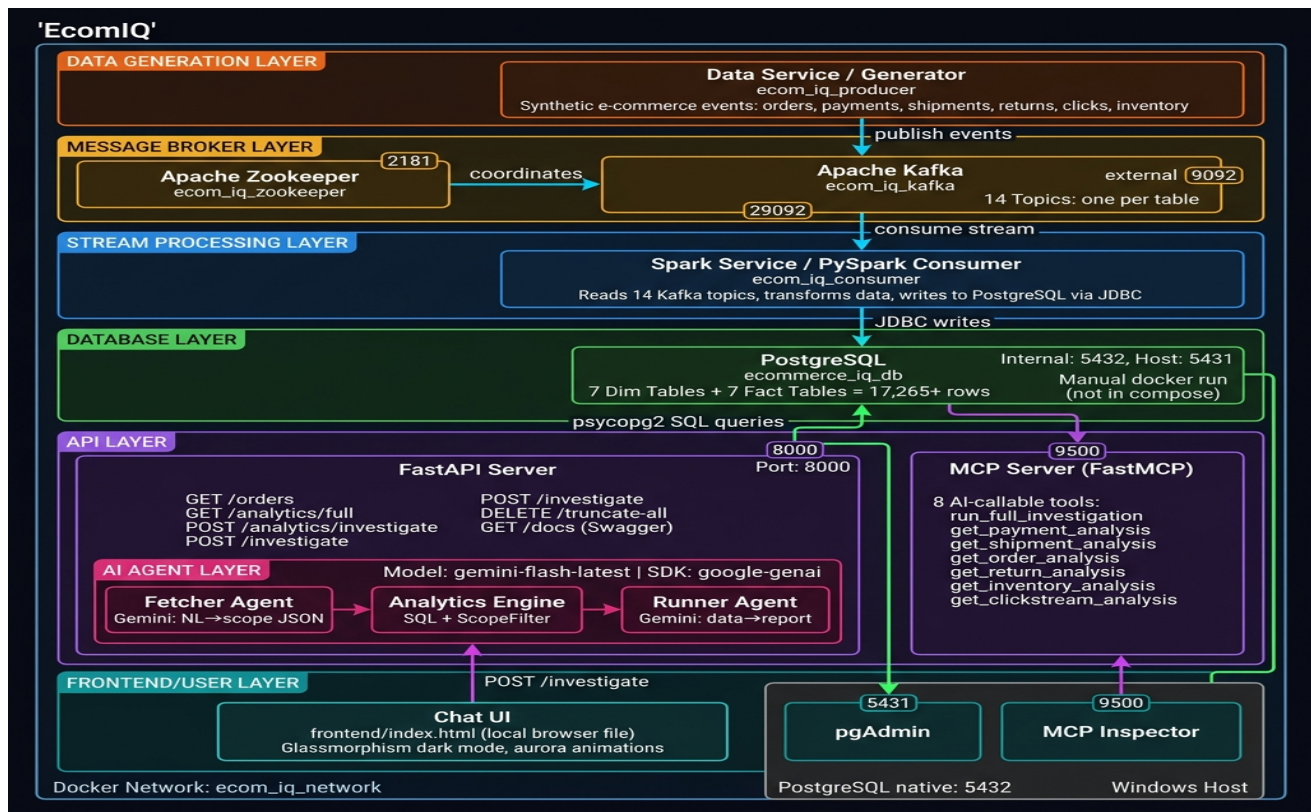


Figure 1.1: EcomIQ End-to-End System Block Diagram showing container networks, ports, and agent pipelines.

2. Container & Module Deep Dive

EcomIQ consists of 6 distinct services operating across data generation, streaming, processing, storage, API serving, and UI rendering.

2.1 Data Service (ecom_iq_producer)

What it does: Acts as the synthetic event generator for the entire system. It runs generator.py and producer.py in an infinite loop. It constructs 14 distinct JSON event payloads representing real-world e-commerce activities: customer profile updates, address registrations, placed orders, line items, payment processing attempts, courier shipments, product returns, warehouse stock transactions, and website clickstream events.

- **Why Synthetic Data?** Allows end-to-end stress testing of streaming pipelines and AI analytics without requiring live production API integrations.
- **Internal Operation:** Uses Python's standard uuid, random, and datetime libraries to deterministically push events to Kafka topics over ecom_iq_kafka:29092.

2.2 Apache Zookeeper (ecom_iq_zookeeper)

What it does: Manages and coordinates the Apache Kafka broker cluster. It maintains cluster metadata, tracks active broker nodes, and handles topic partition leader election.

- **Port 2181:** Exposed internally and on the host for client coordination and health monitoring.

2.3 Apache Kafka (ecom_iq_kafka)

What it does: Serves as the central high-throughput event streaming backbone. It maintains 14 separate Kafka topics (one per database table).

- **Dual Listener Configuration:**

- **Internal Listener (port 29092):** Used by Docker containers (Producer, Spark Consumer, API) on ecom_iq_network.
- **Host Listener (port 9092):** Allows external tools on Windows to inspect topics and consume raw messages.

2.4 Spark Service (ecom_iq_consumer)

What it does: Runs a PySpark structured streaming job (kafka_consumer.py). It subscribes to all 14 Kafka topics, deserializes JSON streams in micro-batches, validates schema structures, and writes transformed records directly into PostgreSQL via JDBC.

- **JDBC Target:** jdbc:postgresql://ecommerce_iq_db:5432/postgres.

2.5 PostgreSQL Database (ecommerce_iq_db)

What it does: The primary relational data warehouse storing 17,250+ historical and real-time records formatted across 14 tables.

- **Schema Breakdown:** Divided into 7 Dimension Tables (reference/lookup) and 7 Fact Tables (transactional).
 - *Dimension Tables:* dim_customers, dim_customer_addresses, dim_regions, dim_skus, dim_categories, dim_payment_methods, dim_warehouses.
 - *Fact Tables:* fact_order_headers, fact_order_lines, fact_payments, fact_shipments, fact_returns, fact_inventory_transactions, fact_user_clickstream.

2.6 API Service & AI Agents (ecom_iq_api)

What it does: A dual-server container running both a REST API (FastAPI on port 8000) and an MCP Server (FastMCP on port 9500). It houses the Multi-Agent AI system:

- **FetcherAgent (agents/fetcher_agent.py):** Uses Gemini AI to extract structured scope parameters (region, payment method, courier, date ranges) from natural language user prompts into JSON.
- **AnalyticsEngine (analytics_engine.py):** Converts scope JSON into parameterized SQL WHERE clauses across all 14 tables, executes queries via psycopg2, and runs a statistical Signal Detector algorithm (flagging anomalies where metrics deviate >1.5x or <0.5x from baseline averages).
- **RunnerAgent (agents/runner_agent.py):** Takes the SQL execution data and signal anomalies, feeding them back to Gemini AI to author a 6-section executive investigation report in markdown.
- **SupervisorAgent (agents/supervisor_agent.py):** Orchestrates the end-to-end flow from query input to final report output.

2.7 Spatial Glass Frontend UI (frontend/)

What it does: A modular, zero-build client interface inspired by iOS liquid glassmorphism and spatial computing designs. It features animated parallax space nebula canvases, glowing lavender focus rings, auto-resizing text input, typing step indicators, and live markdown rendering.

3. Port Mapping & Network Reference

Every network port in EcomIQ is explicitly configured to prevent host port collisions (specifically with native Windows PostgreSQL installations).

Port	Container / Service	Accessibility	Protocol / Role	Why This Specific Port?
2181	ecom_iq_zookeeper	Internal & Host	TCP / Zookeeper Client	Standard default port for Zookeeper cluster coordination.
29092	ecom_iq_kafka	Internal Docker	PLAINTEXT / Broker	Dedicated internal listener preventing IP collision with host OS.
9092	ecom_iq_kafka	Host (localhost)	PLAINTEXT_HOST / Kafka	Standard external Kafka listener port for Windows client tools.
5432	ecommerce_iq_db	Internal Docker	PostgreSQL Database	Default internal PostgreSQL listener for PySpark & FastAPI.
5431	ecommerce_iq_db	Host (localhost)	PostgreSQL Host Map	Crucial: Mapped to 5431 on host to avoid conflict with native Windows PG on 5432.
8000	ecom_iq_api	Host (localhost)	HTTP / FastAPI REST	Primary web server serving /investigate endpoint & Swagger UI.
9500	ecom_iq_api	Host (localhost)	HTTP / FastMCP SSE	Dedicated Model Context Protocol server endpoint for external AI agents.

4. Technology Selection Rationale Matrix

Each technology in EcomIQ was chosen based on specific architectural trade-offs compared to industry alternatives:

Component	Chosen Technology	Alternative Considered	Why Chosen? (Decision Rationale)
Message Streaming	Apache Kafka	RabbitMQ / AWS SQS	Kafka provides log immutability, message replayability, and multi-consumer topic partition scaling required for stream analytics.
Stream Processing	PySpark	Apache Flink / Pandas	PySpark offers fault-tolerant structured streaming, micro-batch windowing, and built-in JDBC writers for PostgreSQL without memory caps.
Data Storage	PostgreSQL	MongoDB / Snowflake	ACID transactional compliance and ultra-fast multi-table SQL JOIN capabilities required for complex root-cause analytics across 14 tables.
API Framework	FastAPI	Flask / Django	Asynchronous ASGI performance (uvicorn), automatic OpenAPI documentation, and strict Pydantic request validation out of the box.
AI Infrastructure	Google Gemini	OpenAI GPT-4 / Claude	Ultra-fast inference latency, large context window for raw SQL dataset inspection, and native structured JSON schema generation.
Agent Tool Protocol	FastMCP	Custom REST / LangChain	Standardized Anthropic Model Context Protocol allowing any external AI agent to inspect and call analytics tools via SSE.

5. Step-by-Step Data & Query Flow

5.1 Real-Time Streaming Data Ingestion Flow

1. **Event Trigger:** `data_service/main.py` invokes `generator.py` to construct synthetic JSON event objects.
2. **Kafka Publish:** `producer.py` sends payloads to designated Kafka topics (e.g., `fact_payments` topic).
3. **Stream Processing:** `spark_service/kafka_consumer.py` reads micro-batches from Kafka via `ecom_iq_kafka:29092`.
4. **DB Persist:** PySpark formats rows and commits inserts to PostgreSQL (`ecommerce_iq_db:5432`).

5.2 Natural Language AI Investigation Flow

1. **User Input:** User types *'Why are payments failing in West India?'* in `frontend/index.html`.
2. **HTTP POST:** Frontend posts JSON payload `{"query": "..."}` to `http://localhost:8000/investigate`.
3. **FastAPI Entry:** `api_service/main.py` passes the prompt to `SupervisorAgent.investigate()`.
4. **Scope Extraction:** `FetcherAgent` calls Gemini API (`gemini-3.6-flash`) to extract: `{"region": "West India", "payment_status": "failed"}`.
5. **SQL Analytics:** `AnalyticsEngine` converts scope into dynamic SQL queries with `WHERE dr.region_name = 'West India'` across all 14 tables, returning aggregated row counts, failure rates, and triggering the statistical signal detector.
6. **Report Generation:** `RunnerAgent` sends SQL metric summaries to Gemini API to compose a structured 6-section markdown report.
7. **UI Render:** FastAPI returns the report JSON, and the Spatial Glass UI renders the report inside an animated floating glass slab.

6. Module Invocation & Operations Manual

6.1 Code Invocation Hierarchy

The table below details how source code files invoke each other during system execution:

Caller Module	Callee Module / Target Function	Interaction Purpose
<code>frontend/js/app.js</code>	<code>frontend/js/api.js</code> → <code>fetchInvestigation()</code>	Initiates HTTP POST request to API endpoint.
<code>api_service/main.py</code>	<code>SupervisorAgent.investigate()</code>	Hands off user prompt to AI orchestration supervisor.
<code>SupervisorAgent</code>	<code>FetcherAgent.extract_scope()</code>	Queries Gemini LLM to extract JSON filter parameters.
<code>SupervisorAgent</code>	<code>AnalyticsEngine.run()</code>	Executes SQL queries against PostgreSQL DB.
<code>AnalyticsEngine</code>	<code>Database.query()</code> via <code>psycpg2</code>	Sends raw SQL string commands to PostgreSQL.
<code>SupervisorAgent</code>	<code>RunnerAgent.generate_report()</code>	Sends SQL data to Gemini LLM to write final report.

6.2 Operations Cheat Sheet

• Start Entire Platform:

```
docker compose up -d
  docker start ecommerce_iq_db
  docker network connect ecom_iq_network ecommerce_iq_db
```

• Switch Gemini AI Models:

Edit model="gemini-3.6-flash" in api_service/agents/fetcher_agent.py and runner_agent.py.
Hot-patch container: docker cp api_service/agents/fetcher_agent.py ecom_iq_api:/app/agents/fetcher_agent.py
and docker restart ecom_iq_api.

• Access User Interfaces:

- **Spatial Glass Chat UI:** Double click frontend/index.html in any web browser.
- **FastAPI Swagger Docs:** Open <http://localhost:8000/docs>.
- **PostgreSQL pgAdmin Connection:** Host localhost, Port 5431, DB postgres, User postgres, Password bibtya1000.